AI506: Advanced Machine Learning

Project 1: Cat & Dog Classification

Table of Contents:

Task 1 - Creating the dataloader

To handle the huge dataset of 3600 jpeg images, we implemented a data loading pipeline using the strategies below.

Data Loader Structure

We utilised torchvision.datasets.ImageFolder as suggested in the project hint. This was efficient as the data was already organised into test, train, and validation directories.

On-the-fly Data Augmentation

On-the-fly data augmentation refers to a technique where modified versions of the training data is created during the training process, instead of creating and saving all the augmented data beforehand.

We applied RandomResizedCrop(), RandomHorizontalFlip(), RandomRotation(), and ColorJitter. These image transformations will train our model to be translation invariant and scale invariant, ensuring that it can still recognise a cat or dog regardless of its position, orientation, or lighting conditions. By slightly altering the images in each epoch, we prevent the model from memorising specific pixel locations, improving generalization. This also helps to increase the amount of training data to prevent overfitting.

Normalization

The pixel values of the training data were normalized using the ImageNet MEAN [0.485, 0.456, 0.406] and STD [0.229, 0.224, 0.225], to ensure that the distribution of input data has zero mean and unit variance. This keeps the initial activations within a range that avoids activation function saturation, hence preventing the possibility of diminishing gradients and ensuring stable weight updates during the initial phases of training.

Shuffling of Input Data

For the input data during training, we set shuffle=True to randomise the order of images in each epoch so that the model doesn't generalize over a specific sequence of classification.

Task 2 - Constructing the network

We designed a Convolutional Neural Network (CNN) named CatDogCNN.

Model Architecture

CatDogCNN utilises the Double-Convolution Blocks DoubleConvBlock where two 3x3 convolutions are applied before each pooling layer. As discussed in the Convolutional Networks lecture, stacking two 3x3 filters gives a receptive field equivalent to one 5x5 filter but with fewer parameters and more non-linearity. Having two layers instead of one allows for the application of ReLU activation function twice, enabling the network to learn much more complex, non-linear features like the specific shapes and curves of animal body parts. Stacking layers also allows the first convolution to find the edges, and the second to

combine them into textures before downsampling via pooling. nn.MaxPool2d(2, 2) reduces the height and width of the images by half, helping the model focus on high-level features.

Model Complexity

We made use of a hierarchical feature extraction pattern. This helps the model break down the complexities by reducing spatial resolution from 128px to 8px. On the other hand, the feature depth expands from 32 to 256. This allows the model to learn low-level features like edges in the early layers, and high-level complex features like distinct animal body parts in the deeper layers that are then condensed via GAP for the final convolutional stage.

Global Average Pooling (GAP)

Instead of flattening the 8 x 8 x 256 feature map into a massive vector, GAP allows us to average each feature map into a single value which greatly reduces the total number of parameters in the classifier, significantly lowering the risk of overfitting.

Activation Function

We used ReLU for all hidden layers and the classifier head. ReLU was highlighted in the Feedforward Networks lecture as a preferred activation function as its non-saturating gradient leads to faster convergence than Sigmoid or Tanh. We believe it provides the necessary non-linearity capabilities for the model to learn complex animal features.

Regularization (Dropout, BatchNorm)

BatchNorm2d is added after every convolution to stabilise and accelerate the training process by normalizing the next layer inputs. Dropout(p=0.5) layer is used in the fully connected classifier to randomly zero out 50% of the neurons during training, preventing the model from becoming overly reliant on specific neurons. This is a form of anti-overfitting discussed in the Regularization lecture.

Optimizer (AdamW)

We selected the AdamW optimizer with a learning rate of 0.001 and ReduceLROnPlateau scheduling. AdamW is able to decouple weight decay from the gradient update for better regularization performance as compared to Adam. The scheduler automatically reduces the learning rate when the model stops improving, a strategy discussed in Hyperparameter Dynamics. This helps the model to fine-tune its weights towards the end of the training.

Tracking Best Performance and Early Stopping Technique

We check whether the model has achieved its best performance after each epoch, by monitoring the validation loss using improved = vl_loss < best_val_loss, and saving the weights if applicable. This helps to keep track of the model's best weights after the training process has ended, not just the latest update. If the model does not improve after 5 consecutive epochs, we exercise early stopping, and stop the training early to prevent the model from memorising the training images and causing overfitting.

Hyperparameter Tuning using Optuna Study

At first, we used these default hyperparameters:

Hyperparameter	Value	Justification
IMG_SIZE	128	We thought 128x128 is large enough to understand key features of cats and dogs while allowing reduced memory and fast training.
BATCH_SIZE	32	We avoided too small or too large batch sizes, so we chose somewhere between 16 and 64 that can speed training and save GPU memory without overfitting.
NUM_EPOCHS	30	Initially, we tried other epochs (5 and 20). The accuracy kept increasing, but the training slowed down, so we kept it at 30 for optimal speed as well as accuracy, and used early stopping along the way.
LR	0.001	This is Adam optimizer's default learning rate. We believe small learning rates stabilize the small batch size.
WEIGHT_DECAY	$1e^{-4}$	Since we have over 1M trainable parameters with only 2K training images, we used weight decay to prevent overfitting. The value $1e^{-4}$ is a penalty to the loss function (L^2 norm), keeping the weights small and preventing a single pixel feature from dominating the decision-making.
PATIENCE	5	For stability and efficiency, we wanted the training to stop after 5 epochs to escape from too long of a local plateau.
NUM_WORKERS	2	Two workers match typical CPU core count for data loading. Higher values can cause CPU overhead.

However, we believed there would be better hyperparameters we could implement. Since this is a neural network model, we decided to tweak the learning rate ($1e^{-5}$ to $1e^{-2}$), batch size (16, 32, 64), weight decay ($1e^{-5}$ to $1e^{-3}$), and patience (3 to 7). Those hyperparameters define how the network learns and generalizes. We kept the number of epochs fixed because the early stopping handled effective epochs. After over an hour of running Optuna study, it turned out that the best hyperparameters were as below with final validation loss of 0.4403.

```
batch_size: 32
lr: 0.00011701416881131436
weight_decay: 0.0005674118614493388
patience: 6
```

Task 3 - Visualization of results

In this section, we analyse the internal representations and performance of our CatDogCNN.

Summary of Training

Before hyperparameter tuning:

- Lowest validation loss: 0.5596 (Epoch 29)
- Test accuracy: 73.00%

Weights from Epoch 29 were saved as the best-performing, and used for final test evaluation on the test dataset.

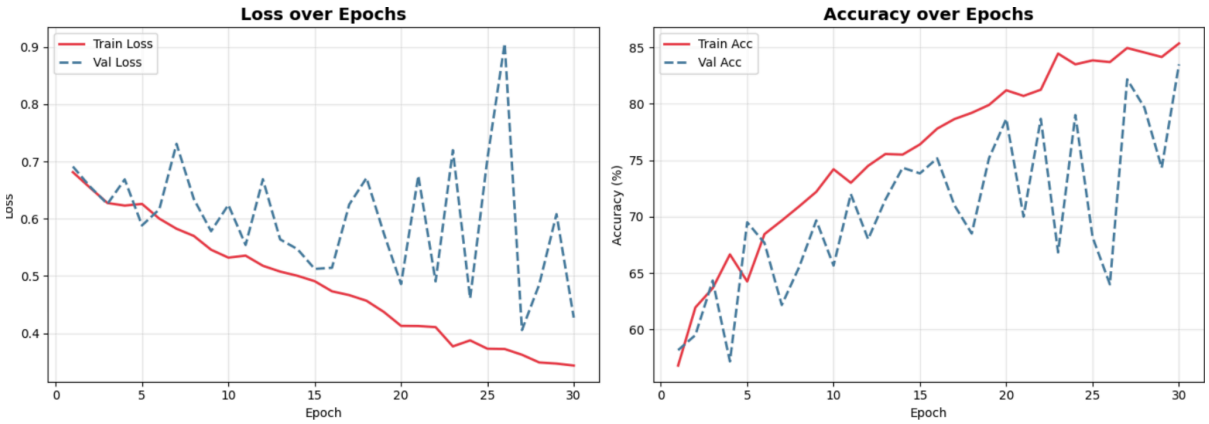
After hyperparameter tuning:

- Lowest validation loss: 0.4056 (Epoch 30)
- Test accuracy: 84.25%

As we can see, CatDogCNN performed better after hyperparameter tuning (accuracy cranked up by 11%, validation loss dropped by ~0.1). However, more epochs could've been implemented for higher accuracy, but sadly we didn't have enough time to train further.

Learning Curves

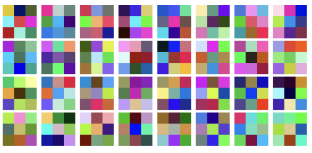
The Loss and Accuracy over 30 Epochs was plotted for both training and validation sets.



First, we observe that the training loss mostly follows a consistent downward trend, while the validation loss, though generally decreasing, exhibits more frequent spikes and volatility. The divergence between training and validation loss, from epoch 15, suggests that the model is reaching its generalization limit although it is successfully learning.

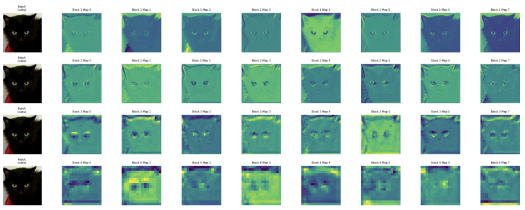
Next, we observe that the validation accuracy follows a similar upward trend with the training accuracy. A generalization gap also emerges around epoch 15. The high oscillations in the validation set graphs are likely due to the difficulty of the task and the small batch size.

Learned First-Layer Filters



The first-layer filters look like edge detectors and colour blobs. Such is the standard hierarchy of a CNN. Low-level features are learned before moving onto more complex ones.

Feature Maps per Convolution Block



We visualised the activations of an input cat image as it passed through the DoubleConvBlock stages.

In blocks 1 and 2, the feature maps still clearly resemble a cat image, focusing on the edges and

texture of the image. In blocks 3 and 4, the image becomes more abstract, and represent high-level semantic features. This shows that the deeper layers are discarding redundant spatial information like the background, to focus on the patterns that define a cat.

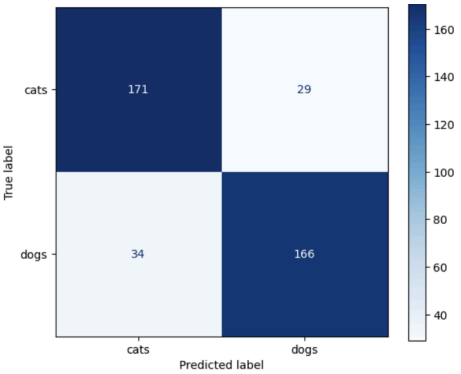
Misclassified Samples

The test loader was shuffled randomly before generating the misclassified samples. This helps to confirm that the shown samples are fairly distributed across both classes.

We observed some patterns in the misclassified images. First, presence of close-up images, where the full animal is not present. Second, images where humans are holding the animals, which can add “noise” to the shape of the animal. Third, images where the animals are in ambiguous poses, such as being curled up or seen from odd angles. Fourth, images of drawn animals, which clearly have different features than real photos. These errors suggest that the model’s weakness is in poses or shapes not heavily represented in the training set.

Confusion Matrix

The confusion matrix shows a balanced performance on both cats and dogs. There are 171 cats and 166 dogs being correctly identified, with 29 cats and 34 dogs being wrongly identified. We can conclude that the model does not have significant bias towards either one of the classes. This is because the dataset used was not imbalanced, and the images were of high quality.



Bonus Task (Testing Pretrained EfficientNet-B0)

The pretrained model EfficientNet-B0 that we chose randomly could achieve a test accuracy of 92.5%, having a better performance than our custom CNN’s 84.25% (after hyperparameter tuning). This comparison highlights the power of transfer learning. While our custom CNN had to learn basic features from a set of limited training images, the EfficientNet-B0 already had a rich understanding of visual features learned from the massive ImageNet dataset. This allowed the pretrained model to reach a higher accuracy.